codegraph_safe.sh — Companion Guide

Revision: 1 **Last modified:** 2026-09-24T12:00:53Z

Source: `constitution/scripts/codegraph/codegraph_safe.sh` (561 lines). Every statement below cites `codegraph_safe.sh:<line>`.

Overview

`codegraph_safe.sh` is the ONLY sanctioned entry point for CodeGraph `init`, `index` and `sync` (line 3). It exists to make the failure modes of the 2026-09-23 bulk-window incident impossible by construction (lines 5-32):

1. Stock `init` / `index` on a fresh DB drops the FK child-key indexes during the bulk window, so `INSERT OR REPLACE` cascades full-scan very large tables. Bulk ops run ONLY through the patched runner derived by `fk_index_patch.py`, or through stock AFTER `fk_cascade_probe.py` proves stock safe. A refused runner is NEVER silently replaced by stock (lines 7-11, 411-437).
2. `codegraph … || true` masking — every failure is a non-zero exit (line 12).
3. A stall that looks alive — `index_watch.py` proves progress; a STALL is exit 7 (lines 13-14, 174-177).
4. Unbounded DB growth — `disk_tripwire.sh` SIGSTOPs the indexer below `--tripwire-min-gib` (exit 8), plus a host preflight (lines 15-16, 170-173, 297-329).
5. The fix vanishing after an upgrade — the runner is re-derived per stock version and its receipt/version checked on every bulk run (lines 17-18, 424-428).
6. The v1.6.0 stale-lock behaviour (a lock older than 2 minutes deleted regardless of holder liveness) — this wrapper checks the lock holder's real `/proc` argv and refuses while it lives, whatever the lock's age (lines 19-22, 331-378).

7. `sync` on any populated DB used to run stock + foreground with no patches, watcher or tripwire. `sync` is now classified BULK vs INCREMENTAL by the pending-refs backlog (lines 23-32, 467-487).

Hard rule: never run a writer against a project root whose lock holder is alive. The script enforces this itself (exit 3), but equally never run stock `codegraph init|index|sync|serve|unlock` by hand on such a root — that bypasses every check below. `codegraph_guard.sh` refuses tracked scripts that do so.

Prerequisites

- `bash`, `flock`, `setsid`, `nohup`, `df`, `python3` (line 102).
- Siblings in the same directory: `codegraph_safe_helper.py`, `index_watch.py`, `disk_tripwire.sh`, `fk_index_patch.py`, `fk_cascade_probe.py` (lines 102-103, 112-116).
- A stock CodeGraph CLI on `PATH` or in `CODEGRAPH_BIN` (line 117).
- For bulk ops: host headroom as measured by `preflight` (see below).

2026 09 25

Host-adaptive V - - , BOB-XXX)

heap budget (added

`preflight` now also reports a `heap_mb=` line: half of currently-measured `MemAvailable` (kB, from `/proc/meminfo`), floored at 8192 MiB and capped at 65536 MiB. The caller (`init` / `index` / `sync`) extracts it and exports `NODE_OPTIONS=--max-old-space-size=$heap_mb` (additive to any pre-existing `NODE_OPTIONS`) before launching the detached supervisor, so the indexer child inherits it. Without this, a stock launch runs on Node's *default* old-space limit (~4 GiB) regardless of host RAM — on a large repo (584K files / 49M edges observed) the resolve phase legitimately needs far more than that and the indexer OOM-aborts (`rc=134`, `FATAL ERROR: Reached heap limit`) *after* finishing file parsing, discarding the whole bulk run (reproduced live 2026-09-25; see `docs/codegraph/Status.md`). Never hardcoded (§12.11) — always derived from the live measurement each run. Extraction failure (an unreadable/missing `heap_mb=` line) refuses the run (`die 6`) rather than silently falling back to the stock/unmeasured limit. Regression-tested: `tests/test_unit_safe.sh` U44 (proportional formula) / U45 (floor). **Known gap, not fixed in this pass:** the line-number citations

elsewhere in this document (§ headers above referencing specific `codegraph_safe.sh` line numbers) were NOT re-walked after this ~20-line insertion and are now stale by that offset for any code below the insertion point — a pre-existing pattern in this doc (see the sibling `codegraph_safe_helper.md`'s own disclosed staleness), tracked as owed follow-up, not silently left unmentioned.

Usage examples

```
# Host check only (no DB touched)
codegraph_safe.sh --project <project_root> preflight

# First full index, detached, return immediately
codegraph_safe.sh --project <project_root> index

# Same, block until the detached run finishes; exit = run result
codegraph_safe.sh --project <project_root> --wait --poll-s 30 index

# Sync; classification decided by the pending-refs backlog
codegraph_safe.sh --project <project_root> sync

# Force the supervised bulk path for sync
codegraph_safe.sh --project <project_root> --bulk sync

# Read-only checks
codegraph_safe.sh --project <project_root> status
codegraph_safe.sh --project <project_root> --expected-count 580000 --tolerance-pct 2 verify

# Remove a lock whose holder is provably dead (decision logged)
codegraph_safe.sh --project <project_root> --reap-dead-lock unlock
```

Operations (lines – , –)

Exactly one op is allowed (line 262); missing or unknown op → exit 2 (lines 265-266).

Op	Kind	Behaviour
`init`	writer	Always bulk. Runner args `init -y -v <P>` (line 515).
`index`	writer	Always bulk. Runner args `index -v <P>` (line 516).
`sync`	writer	BULK or INCREMENTAL, see classification. Bulk runner args `sync <P>` (line 517).
`status`	reader	`lock_check reader` , `/proc` scan (3 tries), then stock `status <P>` ; non-zero stock exit → 1 (lines 449-454).
`verify`	reader	`lock_check reader` , scan, then `verify_now` (helper `verify` + `scope`) → `VERDICT PASS` 0 / `VERDICT FAIL` 1 (lines 455-459, 398-407).
`pre-flight`	other	Prints measurements; `PREFLIGHT PASS` 0 or exit 6 (lines 442-444).
`unlock`	other	Takes the wrapper flock, runs `lock_check unlock` (writer rules); exit 0 when no stale lock remains (lines 445-448).

Writers first take the wrapper flock, then `lock_check writer` , then the `/proc` scan (lines 462-464).

Options (lines - , -)

Option	Default	Validation / notes
`--project DIR`	git toplevel of `$PWD`, else `$PWD` (lines 284-286)	must exist (line 287); path containing whitespace → exit 2 for bulk ops (line 519)
`--bulk`	off	writer ops only, else exit 2 (lines 277-279); only changes behaviour for `sync`
`--pending-threshold N`	`$CG_SAFE_PENDING_BULK_MIN`, else `150000` (line 240)	integer ≥ 1 (lines 271-272)
`--patches IDS`	`$CG_SAFE_PATCHES`, else `fkidx1,resolve1` (line 240)	`all` or comma list `[a-z0-9]`, no leading/trailing/double commas (lines 273-276)
`--wait`	off	block until the detached bulk run writes its result (lines 543-561)
`--poll-s N`	10	integer (lines 267-269)
`--watch-interval N`	60	integer; passed to `index_watch.py`
`--stall-min M`	10	number, decimals allowed (line 270)
`--trip-wire-min-gib N`	20	integer in `[5,500]` (line 280)
`--trip-wire-interval N`	30	integer

Option	Default	Validation / notes
`--reap-dead-lock`	off	allow removal of a provably dead lock
`--expected-count N`	—	integer (line 281); with `--tolerance-pct P` (number, line 282; default 0 when only N given, line 294)
`--status-doc FILE`	`<P>/docs/codegraph/Status.md` (line 292)	absent file → ledger SKIP, run not failed (lines 221-224)
`--scope-baseline FILE`	`scope_baseline.txt` beside the script (line 241)	passed to helper `scope`
`--scope-exceptions FILE`	`scope_exceptions.txt` beside the script	narrow per-exact-path exceptions for the `secret_named` / `secrets_dir` heuristic classes ONLY (never the credential-CONTENT classes); passed as helper `scope`'s optional 3rd argument at every call-site, including inside `supervise()` via env `CG_SV_SCOPE_EXCEPTIONS`; absent file ⇒ no exceptions (unchanged, strictest posture). See `docs/scripts/codegraph_safe_helper.md`'s `### scope` section for the full mechanism.
`-h`, `--help`	—	prints the header, exit 0 (line 260)

Any other `-…` option → exit 2 (line 261). An option missing its value → exit 2 (`need`, line 242).

Patch set (lines –)

Only `fkidx1` and `resolve1` have tests; `lockfix1` and `datafrag1` have none and are opt-in via `--patches` / `CG_SAFE_PATCHES`. `fkidx1` must be in the set — the receipt gate requires it, else exit 5 (lines 45-46). The default runner key is `<version>-fkidx1-resolve1`.

UNCONFIRMED: the receipt check itself (`python3 codegraph_safe_helper.py receipt` , line 425) was not read; the fkidx1 requirement is stated in the header only.

```
Environment (lines      -    )
```

Variable	Default	Effect
`CODEGRAPH_BIN`	`codegraph`	stock CLI (line 117)
`CG_SAFE_PENDING_BULK_MIN`	150000	default sync BULK threshold
`CG_SAFE_PATCHES`	`fkidx1,resolve1`	default `--patches`
`CG_SAFE_PENDING_PROBE_BUDGET_S`	30	wall-clock budget of the pending probe (UNCONFIRMED: consumed inside the helper; not referenced in this script's code)
`CG_SAFE_PATCH_TOOL` / `CG_SAFE_PROBE_TOOL`	sibling `fk_index_patch.py` / `fk_cascade_probe.py`	lines 115-116
`CG_SAFE_MIN_THREAD_HEADROOM`	1024	preflight thread headroom (line 322)
`CG_SAFE_TEST_FREE_BYTES`	—	TEST-ONLY, can only lower free bytes (line 303)
`CG_SAFE_TEST_MEMAVAIL_KB`	—	TEST-ONLY, can only lower MemAvailable (line 314)
`CG_SAFE_TEST_THREADS_USED`	—	TEST-ONLY, can only raise threads used (line 320)

Sync classification: BULK vs INCREMENTAL (lines –)

`BULK=1` by default (line 471). For `sync`:

1. `--bulk` given → BULK (`reason=--bulk requested`, line 474).
2. Else helper `dbinfo` must report the DB exists, nodes present, no error (lines 476-477). If not → BULK (`reason=database missing, empty or unreadable`, line 485).
3. Else helper `pending <P> --threshold N` (line 478):
   - exit 0 → INCREMENTAL (`pending<threshold`, line 480);
   - exit 1 → BULK (`pending>=threshold`, line 481);
   - any other exit → BULK (`pending-probe UNKNOWN`, line 482). UNKNOWN is never read as "small".

Every sync prints `sync classification: BULK|INCREMENTAL (reason=…)`. The header states the pending probe is bounded (an index range read of at most threshold+1 entries, never `COUNT(*)`, lines 29-31); UNCONFIRMED from this file — implemented in `codegraph_safe_helper.py`.

INCREMENTAL path (lines 489-503): stock `sync <P>` in the foreground under the held flock, output to `index_runs/sync_<UTC>.log`, last 5 lines echoed. Non-zero → exit 1. Then helper `verify` + `scope`; ledger entry; `VERDICT PASS` 0 / `VERDICT FAIL` 1. No preflight, no watcher, no tripwire.

Bulk path (lines –)

1. `preflight` — FAIL → exit 6 (lines 505-507).
2. `resolve_runner` (lines 411-437): read stock `--version` (none → 5); run the patch tool with `--patches <IDS> --print-bin` (or `--print-bin` for `all`):
   - exit 0 → runner path must be executable, its parent dir must hold a valid `RUNNER_RECEIPT.json` for the stock version, and the runner's `--version` must equal stock's; any mismatch → 5;
   - exit 2 → 5 (stock is FORBIDDEN as fallback);

- exit 3 (patch not needed) → `fk_cascade_probe.py --work-dir index_runs/probe --json-out index_runs/probe_verdict.json` must exit 0, then the runner is stock; otherwise 5;
- any other exit → 5.

3. Launch detached: `setsid nohup bash <self> __supervise` with state in `CG_SV_*` environment variables (lines 525-529). The caller waits up to 600 × 0.1 s for the indexer to be identified (lines 533-541).
4. Without `--wait`: exit 0 after launch (lines 543-546). With `--wait`: poll every `--poll-s` seconds until `run_<id>.result` exists; exit with the `EXIT=` code it records (lines 547-561).

Supervisor (lines –)

- Starts the runner with the op args, logs to `index_<UTC>.log`, records PIDs.
- Waits up to 300 × 0.1 s for `.codegraph/codegraph.lock` to name a PID whose identity check passes; that PID (else the launched PID) is the indexer (lines 145-157).
- Arms `index_watch.py` and `disk_tripwire.sh` on the identified indexer (lines 158-163); if none identified, logs a WARN and arms neither (line 165).
- While the indexer runs: a `tripwire_<id>.log.TRIPPED` marker → result 8; a last watcher line containing `STALL_` → result 7, indexer left running for inspection (lines 167-179).
- On exit: helper `verify` (with the reported `Indexed N files` count and stock `status` Files count) + helper `scope`; any non-zero among indexer, verify, scope → result 1; ledger append; result written atomically via `.tmp` + `mv` (lines 180-206).

297 329

Preflight thresholds (lines –)

Check	Pass condition
Disk	free on `.codegraph` (else project) ≥ max(60 GiB, 2 × (db + wal bytes))
Memory	`MemAvailable` ≥ 32 GiB
Threads	`ulimit -u` − threads used by this uid ≥ 1024 (skipped when unlimited)

331 396

Unreadable `df` or `MemAvailable` → FAIL (lines 301, 313).

Lock handling (lines –)

- No lock file → proceed (line 334).
- Numeric PID > 1 whose helper `identity` begins `OK`:
  - reader op → continue, unless the holder's op is `init|index|sync` → 3 (lines 345-349);
  - writer / unlock → 3, "whatever the lock's age, it is NOT stale" (line 351).
- Holder dead, non-numeric content, PID ≤ 1, or alive-but-not-codegraph → stale. Reader: WARN and continue (lines 357-360). Writer without `--reap-dead-lock` → 4 (line 363). With it: re-check immediately before removal (live → 3), then remove and append a `REAPED` line to `index_runs/lock_reap.log` (lines 365-377).
- `/proc` scan via helper `scan` (writer 1 try, reader 3 tries 0.5 s apart) → 3 on a live writer (lines 380-390).
- Wrapper flock `.codegraph/.helix_index.flock` taken non-blocking; held by another launcher → 3; cannot open → 2 (lines 392-396).

Exit codes (header lines – , verified against code)

Code	Meaning	Source
0	Success / verified / detached launch without `--wait` / `--help`	260, 443, 448, 454, 458, 499, 545
1	Indexer failed, verification FAILED, stock status failed, supervisor ended without result, unreadable result	453, 459, 496, 502, 554, 560; result 1 at 186-188
2	Usage error / unknown op / bad option / flock file unopenable / whitespace path	242-287, 394, 519
3	Live writer (lock holder or `/proc` scan) or wrapper flock held	348, 351, 369, 387, 395
4	Stale lock present, not reaped	363
5	Safe runner unavailable — stock NEVER used as fallback	415, 423, 426, 428, 430, 433, 436
6	Host preflight failed	444, 507
7	STALL detected (via `--wait`)	result written at 175
8	Disk tripwire tripped (via `--wait`)	result written at 171

Codes 7 and 8 (and a bulk-run 1) reach the caller only through `--wait`, which exits with the recorded `EXIT=` value (line 561).

Files written under `<P>/.codegraph/index_runs/`

File	Written by
`index_<UTC>.log`	bulk runner output (line 522)
`sync_<UTC>.log`	incremental sync output (line 492) — not listed in the header
`run_<id>.pids`	`supervisor=`, `indexer_launch=`, `indexer=`, `watcher=`, `tripwire=` (lines 139-163, 524)
`run_<id>.result`	`EXIT=<n>` + summary, atomic (lines 203-206)
`watch_<id>.log`	`index_watch.py` output (line 159)
`tripwire_<id>.log` (+ `.TRIPPED`)	`disk_tripwire.sh` (line 161)
`supervisor_<id>.log`	detached supervisor stdout/stderr (line 529)
`lock_reap.log`	reaped-lock decisions (line 375)
`probe/`, `probe_verdict.json`	only when the patch tool says NOT_NEEDED (line 432)

`<id>` is `<UTC>_<pid>` (line 521). `watch_state.json` is deleted before each bulk launch (line 523). Also created: `<P>/.codegraph/.helix_index.flock` (line 394).

Edge cases

- `--bulk` with `init` / `index` is accepted and changes nothing (always bulk).
- `unlock` with a stale lock and no `--reap-dead-lock` → exit 4.
- A missing Status doc is not a failure: `LEDGER: SKIP` (line 222).
- If no indexer identity is observed within 30 s, the run continues with no watcher and no tripwire (line 165).

Related scripts

Guide	Relation
codegraph_safe_helper	`identity`, `scan`, `dbinfo`, `pending`, `verify`, `scope`, `receipt` subcommands
disk_tripwire	Free-space watcher armed by the supervisor
index_watch	Progress-proven stall detector
fk_index_patch	Derives the patched runner
fk_cascade_probe	Proves stock safe when no patch is needed
code-graph_runner_patches	Registered runner patches
codegraph_guard	Gate that refuses bypasses of this script
codegraph_scope_guard	Scope proof
scope_render	Scope generator
codegraph_mcp_probe	MCP reachability probe
lumen_verify	Semantic-index health check
gen_lumenignore	Semantic-index scope generator
code-graph_lumen_overview	Overview

Last verified

2026-09-24 — read `codegraph_safe.sh` lines 1-561 in full (in slices); `bash -n codegraph_safe.sh` passed. The script was NOT executed (a live indexer holds the project DB). Helper subcommand internals were not read.